

Clúster boada: Operacions comuns

Contents

1. Descripció del clúster docent boada
2. Accés remot al cluster docent boada
3. Comptes docents
 1. Petició inicial de les comptes
 2. Obtenció de les credencials
4. Grups d'usuaris (accounts)
5. Coordinadors
6. QOS (Quality of Service)
7. Particions
8. Execució de treballs
 1. Executar comandes a una altra partició
 1. Interactiva
 2. Batch
 2. Executar comandes interactives amb X11
 3. Redirigir sortida standard i d'error
 4. Treballs OpenMP
 1. Codi font exemple
 2. Compil·lació
 3. Execució interactiva
 4. Execució en cua
 5. Treballs MPI
 1. Codi font exemple
 2. Compil·lació
 3. Execució interactiva
 4. Execució en cua
 6. Intel MPI
 1. Compil·lació
 2. Execució en cua
 7. Treballs OpenMP + MPI
 1. Codi font exemple
 2. Compil·lació
 3. Execució interactiva
 4. Execució en cua
 8. Fer servir múltiples nodes
 9. Treballs amb GPU
 1. Codi font exemple
 2. Compil·lació
 3. Execució interactiva
 4. Execució en cua
9. Comandes slurm
10. Sessions interactives i límits als nodes
11. Treball amb GPUs
12. Còpia d'arxius entre boada i un equip personal
13. Canvi de contrasenya
14. Programari addicional
 1. Intel oneAPI
15. Control de sessions i treballs executats

Descripció del clúster docent boada

El clúster boada està format per 16 nodes amb les següents característiques:

- Nodes *boada-1*, *boada-2*, *boada-3* i *boada-4*
 - 2 processadors  Intel Xeon E5-5645 a 2.5 GHz

- 6 nuclis per processador
 - 12 fluxos per processador
 - 24 GB de memòria RAM
 - Placa base Intel S5500WB12VR
 - Chipset Intel 5000V
- Nodes *boada-6*, *boada-7* i *boada-8*
 - 2 processadors  Intel(R) Xeon(R) E5-2609 v4 a 1.70GHz
 - 8 nuclis per processador
 - 8 fluxos per processador
 - 32 GB de memòria RAM
 - Placa base Intel S2600WT2R
 - Chipset Intel C612
- Node *boada-9*
 - 1 processador  Intel(R) Xeon(R) CPU E5-1620 v4 a 3.50 GHz
 - 4 nuclis per processador
 - 8 fluxos per processador
 - 32 GB de memòria RAM
 - Placa base Asus X99-E WS/USB 3.1
 - Chipset Intel X99
 - 1 tarja gràfica  NVIDIA GeForce GTX 1080 Ti
- Node *boada-10*
 - 2 processadors  Intel(R) Xeon(R) Silver 4314 a 2.40 GHz
 - 16 nuclis per processador
 - 32 fluxos per processador
 - 128 GB de memòria RAM
 - Placa base GIGABYTE MG62-G41-00
 - 1 accelerador de GPU  NVIDIA GeForce RTX 4090
 - 4 accelerador de GPU  NVIDIA GeForce RTX 3080
- Nodes *boada-11*, *boada-12*, *boada-13* i *boada-14*
 - 2 processadors  Intel(R) Xeon(R) CPU 4210R a 2.40 GHz
 - 10 nuclis per processador
 - 20 fluxos per processador
 - 96 GB de memòria RAM
 - 6 DIMMs DDR4 de 16 GB a 3200 MHz (3 DIMMs per processador).
 - Placa base  Dell Poweredge R640 MLK.
- Node *boada-15*
 - 1 processador  Intel(R) Xeon(R) CPU 4210R a 2.40 GHz
 - 10 nuclis per processador
 - 20 fluxos per processador
 - 64 GB de memòria RAM
 - 4 DIMMs DDR4 de 16 GB a 2933 MHz.
 - Placa base: Supermicro X11SPA-TF
 - Targeta acceleradora  ASUS AI CRL-G116U-P3DF PCIe 3.0x16.
- Nodes *boada-16* i *boada-17*
 - 1 processador  Intel(R) Xeon(R) Silver 4314 CPU a 2.40GHz
 - 16 nuclis per processador

- 32 fluxos per processador
- 64 GB de memòria RAM
 - 4 DIMMs DDR4 de 16 GB a 3200 MHz.
- Placa base:  Dell Poweredge R450.
- Nodes *boada-18*, *boada-19* i *boada-20*
 - 2 processador  Intel(R) Xeon(R) Silver 4314 CPU a 2.40GHz
 - 16 nuclis per processador
 - 32 fluxos per processador
 - 128 GB de memòria RAM
 - 4 DIMMs DDR4 de 32 GB a 3200 MHz.
- Tots el nodes tenen un disc local, on tots els usuaris tenen accés a **/scratch/1/{username}** (substitueix {username} pel vostre nom d'usuari)
- A més, tots els nodes veuen el disc del NAS a **/scratch/nas/1/{username}**

 Per gestionar els diversos treballs que es poden executar, es fa servir el sistema de cues **slurm**:

- El sistema de cues s'encarrega d'anar executant els treballs que indiquen els usuaris en funció de l'ús del clúster i dels treballs que demana cada usuari.
- D'aquesta forma, si un usuari llença 1000 treballs i posteriorment un altre usuari llença un parell de treballs, aquest darrer *no* s'haurà d'esperar a que acabin els 1000 treballs que ha llençat el primer usuari.
- S'estableix un ordre en funció de l'ús que demana cada usuari i dels recursos disponibles.

Accés remot al cluster docent boada

Per fer servir aquest cluster cal establir una connexió *SSH* i activar el *X11 forwarding* (heu de substituir {username} pel vostre nom d'usuari):

```
ssh -X {username}@boada.ac.upc.edu
```

 Requisits del sistema operatiu:

- Els sistemes operatius Unix/Linux habitualment ja duen el client SSH.
- En els sistemes operatius Windows cal instal·lar  Cygwin o similar (durant la instal·lació cal seleccionar els paquets **openssh** i **openssl**).
- En els sistemes operatius Mac cal instal·lar  XQuartz.

Comptes docents

Cada quadrimestre es realitza de forma automàtica la baixa i l'alta de les comptes docents de mode que ja estiguin preparades per a cada inici de quadrimestre.

Es crea un nombre superior a les comptes que s'utilitzen habitualment per tal de tenir comptes de *recanvi* en cas de problemes.

Petició inicial de les comptes

 Aquest pas, sols caldrà que el sol·liciteu per a noves assignatures que encara no existeixen a boada.

- Cal que ho notifiqueu al  LCAC, indicant:
 - inicials de l'assignatura
 - el grups de docència que necessiteu (ex: 10,20..)
 - el nombre de comptes que necessiteu per a cada grup de docència (ex: 30)
 - comptes d'usuaris del professorat que ha de tenir accés a les credencials de les comptes.

Obtenció de les credencials

1. Quan el vostre usuari consta com a autoritzat per veure les credencials, trobareu al vostre HOME de boada un directori anomenat *PASSWORDS-{ASSIGNATURA}*, on *{ASSIGNATURA}* correspon a les inicials de l'assignatura en minúscules.
2. Dins d'aquest directori, trobareu el script *showPasswords.sh* on haureu d'usar el password *{ASSIGNATURA}{ANY_ACTUAL}*, on *{ASSIGNATURA}* correspon a les inicials de l'assignatura en minúscules i *{ANY_ACTUAL}* correspon a l'any actual en format de 4 dígits (no confondre el curs actual) per obtenir el llistat. Copieu les dades que necessiteu i **no guardeu les dades desencryptades a boada**. Durant tot el quadrimestre podeu tornar a consultar aquestes dades en cas de necessitat.

Per exemple, per a l'assignatura LCA on s'han demanat 5 comptes per als grups 10 i 20, i *usuari_pdi* correspon al compte autoritzat per aquesta assignatura:

```
usuari_pdi@boada-1:~/PASSWORDS-lca$ ./showPasswords.sh
Teclegeu el password d'encryptat...
enter aes-256-cbc decryption password:
lca1001:xFlZKqsaoGB2
lca1002:cXdcAUietnv3
lca1003:Vl53KdTs2m6X
lca1004:bUF7NbiaS4mc
lca1005:wVUmy3zmTTO1
lca2001:isha7naeT7oj
lca2002:queiQuaex5Ae
lca2003:Tah5liey9Cho
lca2004:Ai5tip4DioKu
lca2005:ouZai4joophe
Agafeu les dades imprescindibles de les comptes i no guardeu les dades
desencryptades a boada.
```

Grups d'usuaris (accounts)

Cada usuari pertany almenys a un *account*.

Account	Observacions
default	Tots els usuaris pertanyen com a mínim a aquest <i>account</i>
execution2	Sols els usuaris autoritzats pertanyen a aquest <i>account</i>

cuda	Sols els usuaris autoritzats pertanyen a aquest <i>account</i>
cudabig	Sols els usuaris autoritzats pertanyen a aquest <i>account</i>
cuda9	Sols els usuaris autoritzats pertanyen a aquest <i>account</i>
compile	Sols els usuaris autoritzats pertanyen a aquest <i>account</i>
iacard	Sols els usuaris autoritzats pertanyen a aquest <i>account</i>

Coordinadors

Podem definir que un usuari per a que actuï com coordinador d'un *account*, això permetrà a l'usuari per exemple, cancel·lar treballs d'altres usuaris.

QOS (Quality of Service)

Amb s'ha definit límits de temps i/o recursos per a treballs de slurm

QOS	Limits	Observacions
interactive	temps màxim: 1 dia	
execution	temps màxim: 10 minuts	
execution2	temps màxim: 30 minuts	
cuda3080	temps màxim: 15 minuts, gpu mínim: 1, gpu màxim: 4	
cuda4090	temps màxim: 15 minuts, gpu mínim: 1, gpu màxim: 1	
cudabig3080	temps màxim: 8 hores, gpu mínim: 1, gpu màxim: 4	
cudabig4090	temps màxim: 8 hores, gpu mínim: 1, gpu màxim: 1	
cuda9	temps màxim: 15 minuts, gpu mínim: 1, gpu màxim: 1	
compileshort	temps màxim: 5 minuts	
compilelong	temps màxim: 15 minuts	
iacard	temps màxim: 1 dia	

Particions

Els nodes del cluster estan distribuïts en diferents particions. Els diferents *accounts* podran accedir a les particions que tinguin assignades:

Partició	Account	Nodes	QOS per defecte	Generic Resources	Observacions
----------	---------	-------	-----------------	-------------------	--------------

				(GRES)	
interactive	default	boada-[18-20]	interactive		Quan accedim al clúster, serem redirigits al node d'aquesta partició amb una sessió interactiva
execution	default	boada-[11-14]	execution		
cuda	cuda	boada-[10]	cuda3080	gpu [gpu:rtx4090:1] [gpu:rtx3080:4]	Partició restringida a usuaris autoritzats
cuda	cudabig	boada-[10]	cudabig3080	gpu [gpu:rtx4090:1] [gpu:rtx3080:4]	Partició restringida a usuaris autoritzats
execution2		boada-[6-8]	execution2		Partició restringida a usuaris autoritzats
cuda9	cuda9	boada-[9]	cuda9	gpu [gpu:gtx1080]	Partició restringida a usuaris autoritzats
compile	compile	boada-[14-17]	compileshort		Partició restringida a usuaris autoritzats
iacard	iacard	boada-[15]	iacard		Partició restringida a usuaris autoritzats

Execució de treballs

Els treballs s'han d'enviar a les cues (en el cas de *boada*, equivalent a les particions) d'execució de clúster:

Cua	Descripció	Observacions
interactive	Per defecte, quan ens connectem tindrem una sessió interactiva en aquesta cua	Els treballs enviats compartiran els recursos de CPU i memòria amb la resta de treballs

execution	La cua que s'ha d'utilitzar com a primera opció	Els treballs enviats no compartiran els recursos de CPU i memòria amb la resta de treballs (exclusiva)
execution2	Una cua especial que usa els nodes <i>boada-[1-4]</i>	Els treballs enviats no compartiran els recursos de CPU i memòria amb la resta de treballs (exclusiva). Aquesta cua és d'accés restringit
cuda3080	Una cua especial que fa servir les GPU RTX3080 del node <i>boada-10</i>	Aquesta cua és d'accés restringit
cuda4090	Una cua especial que fa servir les GPU RTX4090 del node <i>boada-10</i>	Aquesta cua és d'accés restringit
cudabig3080	Una cua especial que fa servir les GPU RTX3080 del node <i>boada-10</i>	Aquesta cua és d'accés restringit
cudabig4090	Una cua especial que fa servir les GPU RTX4090 del node <i>boada-10</i>	Aquesta cua és d'accés restringit
cuda9	Una cua especial que fa servir les GPU del node <i>boada-9</i>	Aquesta cua és d'accés restringit
compileshort	Una cua especial que es fa servir per executar compilacions curtes	Aquesta cua és d'accés restringit
compilelong	Una cua especial que es fa servir per executar compilacions curtes	Aquesta cua és d'accés restringit
iacard	Una cua especial que fa servir la tarjeta d'IA al node <i>boada-15</i>	Aquesta cua és d'accés restringit

- Un treball (job) no és més que un script que indica al cluster les accions a realitzar. Al principi del job es posen les directives (comencen amb #SBATCH) i a continuació les comandes que volem que s'executin. Exemple:

```
#!/bin/bash

#SBATCH --chdir=/scratch/nas/1/{username}

hostname
```

💡 Substitueu **{username}** pel vostre nom d'usuari.

Els jobs es poden executar de dues formes:

- Interactiva: per executar de forma immediata, pensat per realitzar proves abans d'encuar el job. Com que ja estem en una sessió interactiva, podem executar directament l'script:

```
./run-xxx.sh
```

- Sistema de cues: per encuar el job i executar-se quan hi hagi els recursos disponibles. Farem servir la comanda *sbatch* indicant el nom del fitxer a executar:

```
sbatch [-p particio] ./submit-xxx.sh
```

 Si no s'especifica la partició serà la cua **execution**

Executar comandes a una altra partició

Interactiva

Per obrir una sessió interactiva en un node d'una partició especial, farem:

```
salloc -A {account} -p {partition} srun --pty /bin/bash
```

on **{account}** correspon al nom del compte (per exemple: **cuda9**) i **{partition}** correspon al nom de la partició (per exemple: **cuda9**).

Batch

Per enviar un job a una cua que no sigui la *execution*, haurem de definir el fitxer del job:

```
#SBATCH -A {account}
#SBATCH -p {partition}
....
```

on **{account}** correspon al nom del compte (per exemple: **cuda9**) i **{partition}** correspon al nom de la partició (per exemple: **cuda9**).

Executar comandes interactives amb X11

Si ens interessa accedir de forma interactiva i poder executar comandes que fan servir X11, farem:

```
/usr/local/bin/srun.x11 -A {account} -p {partition} [additional params]
```

per exemple per accedir a partició cuda9 farem:

```
/usr/local/bin/srun.x11 -A cuda9 -p cuda9 --gres=gpu:1
```

Redirigir sortida standard i d'error

Per defecte la sortida standard i d'error va al home que tinguem definit amb nom `slurm-$ID.out` i `slurm-$ID.err`, on `$ID` és el número de job.

Si volem que aquests fitxers vagin a un fitxer concret, podem fer servir les opcions `--output` i `--error` de la comanda *sbatch*, o bé posar-ho com a directives:


```
#!/bin/bash

#SBATCH --chdir=/scratch/nas/1/{username}
#SBATCH --output=/scratch/nas/1/{username}/sortida-%j.out
#SBATCH --error=/scratch/nas/1/{username}/error-%j.out

hostname
```

💡 Substitueu `{username}` pel vostre nom d'usuari.

📘 Als noms es poden posar directives per indicar paràmetres d'slurm, a l'exemple `%j` indicarà el JobID. Altres directives:

- `%%`: El caràcter `%`
- `%a`: Job array ID
- `%j`: JobID
- `%N`: Nom del host on s'executa l'script

📘 Es poden veure més directives a l'apartat *filename pattern* del 🌐 manual de la comanda `sbatch`

Treballs OpenMP

Codi font exemple

hello_openmp.c

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[])
{
    int nthreads, tid;
    /* Fork a team of threads giving them their own copies of variables */
    #pragma omp parallel private(nthreads, tid)
    {
        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);
        /* Only master thread does this */
        if (tid == 0)
        {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }
    } /* All threads join master thread and disband */
}
```

Compil·lació

```
icc -fopenmp -o hello_openmp hello_openmp.c
```

⚠️ Per usar `icc` cal haver inicialitzat l'entorn Intel Parallel Studio XE

Execució interactiva

```
$ export OMP_NUM_THREADS=4
$ ./hello_omp
Hello World from thread = 0
Number of threads = 4
Hello World from thread = 3
Hello World from thread = 2
Hello World from thread = 1
```

Execució en cua

hello_openmp.sbatch

```
#SBATCH --nodes=1          ### Number of Nodes
#SBATCH --cpus-per-task=4  ### Number of threads per task (OMP threads)
#SBATCH --chdir=/scratch/nas/1/{YOUR_USERNAME}
#SBATCH --output=/scratch/nas/1/{YOUR_USERNAME}/sortida-%j.out
#SBATCH --error=/scratch/nas/1/{YOUR_USERNAME}/error-%j.out

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
./hello_omp
```

```
$ sbatch hello_openmp.sbatch
Submitted batch job 850
```

Obtindrem de sortida al fitxer `/scratch/nas/1/{username}/sortida-850.out`

```
$ cat /scratch/nas/1/$USER/sortida-850.out
Hello World from thread = 3
Hello World from thread = 0
Number of threads = 4
Hello World from thread = 1
Hello World from thread = 2
```

⚠ En alguns casos, és possible que s'hagi d'afegir el source de les variables al `parallel_studio` per evitar problemes entre llibreries:

```
source /Soft/intel_parallel_studio_xe/parallel_studio_xe_2018/
psxevars.sh
```

Treballs MPI

Codi font exemple

hello_mpi.c

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    char processor_name[MPI_MAX_PROCESSOR_NAME];
```

```

int name_len;
MPI_Get_processor_name(processor_name, &name_len);
printf("Hello world from processor %s, rank %d"
      " out of %d processors\n",
      processor_name, world_rank, world_size);
MPI_Finalize();
}

```

Compil·lació

```
mpicc.mpich -o hello_mpi hello_mpi.c
```

Execució interactiva

```

$ srun --mpi=pmi2 ./hello_mpi
Hello world from processor boada-1, rank 0 out of 1 processors

```

Execució en cua

hello_mpi.sbatch

```

#SBATCH --chdir=/scratch/nas/1/{username}
#SBATCH --output=/scratch/nas/1/{username}/sortida-%j.out
#SBATCH --error=/scratch/nas/1/{username}/error-%j.out
#SBATCH --job-name="testmpi"
#SBATCH --nodes=3
#SBATCH --ntasks=3
srun --mpi=pmi2 ./hello_mpi

```

```

$ sbatch hello_mpi.sbatch
Submitted batch job 876

```

Obtindrem de sortida al fitxer `/scratch/nas/1/{username}/sortida-876.out`

```

$ cat /scratch/nas/1/$USER/sortida-876.out
Hello world from processor boada-2, rank 0 out of 3 processors
Hello world from processor boada-3, rank 1 out of 3 processors
Hello world from processor boada-4, rank 2 out of 3 processors

```

Intel MPI

Compil·lació

```
mpiicc -o hello_mpi hello_mpi.c
```



Ens hem d'assegurar que tenim indicat a la variable `PATH` on es troba el compilador **mpiicc**.

Execució en cua

S'ha de definir la variable **FI_PROVIDER**:

```
export FI_PROVIDER=sockets
```

 <https://www.intel.com/content/www/us/en/developer/articles/technical/mpi-library-2019-over-libfabric.html>

hello_mpi.sbatch

```
#SBATCH --chdir=/scratch/nas/1/{username}
#SBATCH --output=/scratch/nas/1/{username}/sortida-%j.out
#SBATCH --error=/scratch/nas/1/{username}/error-%j.out
#SBATCH --job-name="testmpi"
#SBATCH --nodes=3
#SBATCH --ntasks=3

export FI_PROVIDER=sockets

srun --mpi=pmi2 ./hello_mpi
```

Treballs OpenMP + MPI

Codi font exemple

hello_hybrid.c

```
#include <stdio.h>
#include <omp.h>
#include <mpi.h>

int main (int argc, char *argv[])
{
    int nthreads, thread_id;
    int id, np;
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int processor_name_len;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Get_processor_name(processor_name, &processor_name_len);

    #pragma omp parallel private(nthreads, thread_id)
    {
        thread_id = omp_get_thread_num();
        printf("Hello World from thread %d, process %d of %d, hostname\n",
               thread_id, id, np, processor_name);

        if (thread_id == 0)
        {
            nthreads = omp_get_num_threads();
            printf("Thread %d on process %d of %d reports: nthreads =\n",
                   thread_id, id, np, nthreads);
        }
    }

    MPI_Finalize();
    return 0;
}
```

```
}
```

Compil·lació

```
mpicc -o hello_hybrid hello_hybrid.c -fopenmp
```

Execució interactiva

```
$ export OMP_NUM_THREADS=4
$ srun --mpi=pmi2 ./hello_hybrid
Hello World from thread 0, process 0 of 1, hostname boada-1
Hello World from thread 1, process 0 of 1, hostname boada-1
Thread 0 on process 0 of 1 reports: nthreads = 4
Hello World from thread 3, process 0 of 1, hostname boada-1
Hello World from thread 2, process 0 of 1, hostname boada-1
$ srun --mpi=pmi2 ./hello_hybrid
Hello World from thread 0, process 0 of 1, hostname boada-1
Thread 0 on process 0 of 1 reports: nthreads = 4
Hello World from thread 1, process 0 of 1, hostname boada-1
Hello World from thread 2, process 0 of 1, hostname boada-1
Hello World from thread 3, process 0 of 1, hostname boada-1
```

Execució en cua

hello_hybrid.sbatch

```
#SBATCH --chdir=/scratch/nas/1/{username}
#SBATCH --output=/scratch/nas/1/{username}/sortida-%j.out
#SBATCH --error=/scratch/nas/1/{username}/error-%j.out
#SBATCH --job-name="hybrid"
#SBATCH --nodes=3          ### Number of Nodes
#SBATCH --ntasks=3         ### Number of MPI tasks
#SBATCH --cpus-per-task=4   ### Number of threads per task (OMP threads)
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun --mpi=pmi2 ./hello_hybrid
```

```
$ sbatch hello_hybrid.sbatch
Submitted batch job 900
```

Obtindrem de sortida al fitxer `/scratch/nas/1/{username}/sortida-900.out`

```
$ cat /scratch/nas/1/$USER/sortida-876.out
Hello World from thread 0, process 1 of 3, hostname boada-3
Thread 0 on process 1 of 3 reports: nthreads = 4
Hello World from thread 2, process 1 of 3, hostname boada-3
Hello World from thread 3, process 1 of 3, hostname boada-3
Hello World from thread 0, process 0 of 3, hostname boada-2
Hello World from thread 0, process 2 of 3, hostname boada-4
Thread 0 on process 2 of 3 reports: nthreads = 4
Hello World from thread 2, process 2 of 3, hostname boada-4
Thread 0 on process 0 of 3 reports: nthreads = 4
Hello World from thread 1, process 0 of 3, hostname boada-2
Hello World from thread 3, process 0 of 3, hostname boada-2
Hello World from thread 1, process 2 of 3, hostname boada-4
Hello World from thread 1, process 1 of 3, hostname boada-3
Hello World from thread 3, process 2 of 3, hostname boada-4
Hello World from thread 2, process 0 of 3, hostname boada-2
```

Fer servir múltiples nodes

Podem indicar que el nostre job ha d'executar-se en múltiples nodes amb el paràmetre **--nodes**:

```
sbatch --nodes=4 testmulti.sh
```



Com més nodes demanem, menys prioritat tindrà el nostre job i més trigarà en començar a executar-se.

Quan s'executa un job demanant múltiples nodes, l'execució comença a un dels nodes que s'ha assignat al job i tenim la resta de nodes reservats. Per a poder accedir als diferents nodes des del node inicial, es fa servir la comanda **srun**. Veiem un exemple:

```
#!/bin/bash

#SBATCH --chdir=/scratch/nas/1/{username}
#SBATCH --job-name="testMultiNode"
#SBATCH --wait-all-nodes=1

DIR=/scratch/nas/1/{username}

JOBMASTER="$DIR/master.sh"
JOBSLAVE="$DIR/slave.sh"

for node in `scontrol show hostnames $SLURM_JOB_NODELIST`; do
    if [ "$HOSTNAME" = "$node" ] ; then
        $JOBMASTER &
    else
        srun --nodes=1 --nodelist $node --ntasks=1 $JOBSLAVE &
    fi
done

wait
```



Substitueu **{username}** pel vostre nom d'usuari.

- Al principi de l'exemple hi ha un conjunt de directives:
 - La directiva **chdir** indica el path on s'ha d'executar l'script.
 - La directiva **job-name** indica el nom del job. Ens ajuda a identificar-lo.

- La directiva **wait-all-nodes** indica que el job no començarà fins que no tinguem tots els nodes disponibles.
- Seguidament es preparen un conjunt de variables (DIR, JOBMASTER, JOBSLAVE).
- A continuació es fa un bucle (for) en funció del valor de la variable **SLURM_JOB_NODELIST**. Sempre que demanem un job amb múltiples nodes (paràmetre --nodes), es pot consultar en el job els nodes que se'ns ha assignat mitjançant aquesta variable.
 -  Com el format en que la variable està indicat no és una llista de nodes, es fa servir la crida a **scontrol show hostnames \$SLURM_JOB_NODELIST** que retorna una llista dels nodes indicats separats per espais.
- A l'exemple, dins del bucle mirem si és el node inicial (al que anomenem master) o un dels altres nodes (slaves).
 - Si és el master executem el procés que es considera que s'ha d'executar en aquest node (ho fem en batch posant el símbol & o l'script no continuarà fins que no finalitzi aquest procés).
 - Si és un dels slaves, mitjançant la comanda **srun** executem el procés en el node indicat. Fixem-nos en els paràmetres:
 - --nodes=1 indica que volem executar el proces \$JOBSLAVE en un dels nodes.
 - --nodelist \$node indica el node en el que es vol fer l'execució (recordem que tenim reservats aquests nodes).
 - --ntaks=1 indica que només s'ha d'executar una tasca en el node.
 - \$JOBSLAVE és l'script que s'ha d'executar al node slave.
 - De nou important veure que l'executem en batch mitjançant el símbol &.
- Quan finalitza el bucle fem un wait per esperar a que tots els processos finalitzin.

Treballs amb GPU

Codi font exemple

hello_cuda.cu

```
#include <stdio.h>

__global__ void helloFromGPU (void)
{
    printf("Hello World from GPU!\n");
}

int main(void)
{
    // hello from cpu
    printf("Hello World from CPU!\n");
    helloFromGPU <<<1, 5>>>();
    cudaDeviceReset();
    return 0;
}
```

Compil·lació

```
nvcc -ccbin gcc-4.8 -o hello_cuda hello_cuda.cu
```

Execució interactiva

```
boada-1$ salloc -A {account} -p {partition} --gres=gpu:1 srun --pty /
bin/bash
salloc: Granted job allocation 1137
boada-9$ echo $CUDA_VISIBLE_DEVICES
1
boada-9$ ./hello_cuda
Hello World from CPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
boada-9$ exit
```

💡 on **{account}** correspon al nom del compte (per exemple: **cuda9**) i **{partition}** correspon al nom de la partició (per exemple: **cuda9**).

📘 Amb el paràmetre **--gres** indiquem els recursos de GPU que volem reservar amb la sintaxis **--gres=gpu:[tipus]:quantitat**. En aquest exemple hem reservat una (1) GPU sense especificar quin tipus. Podem veure els diferents tipus de GPU a la taula de particions.

📘 La variable **CUDA_VISIBLE_DEVICES** ens indica el ID de les GPU's que tenim assignades.

Execució en cua

hello_cuda.sbatch

```
#SBATCH --chdir=/scratch/nas/1/{username}
#SBATCH --output=/scratch/nas/1/{username}/sortida-%j.out
#SBATCH --error=/scratch/nas/1/{username}/error-%j.out
#SBATCH --job-name="cuda"
#SBATCH -A cuda9
#SBATCH -p cuda9
#SBATCH --gres=gpu:1
echo "CUDA_VISIBLE_DEVICES=$CUDA_VISIBLE_DEVICES"
./hello_cuda
```

```
$ sbatch hello_cuda.sbatch
Submitted batch job 1196
```

Obtindrem de sortida al fitxer **/scratch/nas/1/{username}/sortida-1196.out**

```
$ cat /scratch/nas/1/$USER/sortida-1196.out
CUDA_VISIBLE_DEVICES=1
Hello World from CPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
```

Comandes slurm

- Per veure la llista dels jobs en espera per executar-se:

```
squeue -t PENDING
```

- Veure la llista dels jobs en execució:

```
squeue -t RUNNING
```

- Veure l'estat detallat d'un job en execució:

```
scontrol show jobid -dd {jobid}
```

- Veure només els nostres treballs:

```
squeue -u $USER
```

- Cancel·lar un treball:

```
scancel {jobid}
```

- Veure l'estat general del cluster:

```
sinfo
```

- Veure l'estat d'un node concret:

```
scontrol show node boada-2
```

- Veure la prioritat dels jobs:

```
sprio -l
```

Sessions interactives i límits als nodes

Per garantir que ningú fa un ús indiscriminat dels nodes, tots estan limitats a un màxim de 5 minuts de CPU per sessió si s'accedeix per **ssh**. Hauria de ser temps suficient per editar, fer petites compilacions i consultar arxius.

Treball amb GPUs

Si es té autorització per treballar amb GPUs, podem entrar al node corresponent indicant el número de GPUs disponibles:

```
salloc -A cuda -p cuda --qos=cuda3080 --gres=gpu:rtx3080:2 srun --pty /bin/bash
```

 Amb el paràmetre `--gres=gpu:rtx3080:2` indiquem el número de GPUs que volem disponibles pel nostre job dins dels límits que permet la cua on executem.

Com els nodes poden tenir diversos tipus de GPUs, es pot indicar el tipus de GPU que es vol

indicant-lo al mateix paràmetre:

```
salloc -A cuda -p cuda --qos=cuda4090 --gres=gpu:rtx4090:1 srun --pty /  
bin/bash
```

O si volem diverses GPUs d'un model concret:

```
salloc -A cuda -p cuda --qos=cuda3080 --gres=gpu:rtx3080:2 srun --pty /  
bin/bash
```

Si es vol accedir a les cues de llarga duració, s'ha d'especificar l'account concret:

```
salloc -A cudabig -p cuda --qos=cudabig3080 --gres=gpu:rtx3080:4 srun  
--pty /bin/bash
```

Còpia d'arxius entre boada i un equip personal

Es poden copiar arxius des d'un equip personal fent servir la comanda **scp** indicant el nom d'usuari:

- Copiar un arxiu a boada:

```
scp file user@boada.ac.upc.edu:
```

- Copiar un arxiu des de boada:

```
scp user@boada.ac.upc.edu:file .
```

 En cas que estiguem fent servir a l'equip personal una versió molt moderna de SSH, és necessari afegir l'opció **-O** donat que la versió actual de SCP de boada no admet el protocol intern SFTP per defecte:

```
scp -O file user@boada.ac.upc.edu:
```

Canvi de contrasenya

Per a poder canviar la contrasenya cal executar exactament:

```
ssh -t usuari@boada.ac.upc.edu passwd
```

 usuari: és l'username al DAC, de la persona.

 La paraula "passwd" és part de la comanda, no la contrasenya.

 L'opció **-t** és necessària perquè el servidor d'entrada no és cap dels nodes del clúster.

Programari addicional

Intel oneAPI

💡 És important que recordeu de connectar a **boada** amb el *X11 forwarding* activat per poder disposar de l'entorn gràfic de la *suite*.

1. Inicialitzem l'entorn:

```
% source /Soft/intel_oneapi/2023_02/setvars.sh
```

2. Ja tenim totes les utilitats disponibles; per exemple:

```
% icc -V
Intel(R) C Intel(R) 64 Compiler Classic for applications running
on Intel(R) 64, Version 2021.10.0 Build 20230609_000000
Copyright (C) 1985-2023 Intel Corporation. All rights reserved.

% ifort -V
Intel(R) Fortran Intel(R) 64 Compiler Classic for applications
running on Intel(R) 64, Version 2021.10.0 Build 20230609_000000
Copyright (C) 1985-2023 Intel Corporation. All rights reserved.
```

Control de sessions i treballs executats

Existeix un mecanisme per a poder veure les dades de connexions i treballs a les cues de les comptes d'assignatures.

Atès que l'adreça IP és una dada personal, no es poden indicar les dades de les IP's de les comptes si no s'ha notificat amb anterioritat als estudiants que es farà aquesta cessió i tractament. Per això s'ha "amagat" les dades d'IP amb un valor hash, de mode que una mateixa IP donarà un mateix valor de hash. A més per facilitar la interpretació de les dades, s'ha indicat cada valor de hash a quin tipus de IP pertany. Per tant tindrem:

```
INFO: Tipologia adreces IPencriptades:
(IP1) IP publica UPC (per ex. aules informàtiques) [147.83.xxx.xxx]
(IP2) IP privada UPC (per ex. eduroam, xarxes NAT, etc.)
[10.xxx.xxx.xxx]
(IP3) IP externa UPC (també pot estar darrera de NAT externa, per ex.
biblioteques, etc.)
```

L'informe de dades que es pot consultar per a un compte inclourà les dades de tot el quadrimestre actual. Per a quadrimestre de tardor tindrem les dades des de setembre fins gener del següent any. I per a quadrimestre de primavera tindrem les dades des de febrer fins agost. Per tant, a dia 1 de setembre i 1 de febrer és quan es fa la neteja d'aquestes dades. Aquestes dades aniran ordenades per la data d'inici de sessió de forma ascendent.

L'execució de script de consulta serà de la següent manera:

```
boada-1:~$ ./LOG-{assignatura}/showUsage.sh
usage: ./showUsage.sh [[-u usuari ] [-i ] [-j ] | [-h]]
```

on {assignatura} correspon a les sigles de l'assignatura.

Per ex:

```
boada-1:~/LOG-par$ ./showUsage.sh -u par4206
```

IMPORTANT: les dades son actualitzades cada 5 minuts! Les connexions inferiors a 5 minuts respecte l'hora actual poden no apareixer encara!

Llistat de sessions de l'usuari par4206 des de Feb-2021 fins Feb-2021:

par4206	pts/9	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:18:16	2021	- Wed Feb 17 17:20:20 2021	(00:02)
par4206	pts/13	(IP1)7a54ad6cb280f0184c0a15bc66a8ffe0	-	Wed
Feb 17	17:19:14	2021	- Wed Feb 17 17:19:14 2021	(00:00)
par4206	pts/9	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:20:32	2021	- Wed Feb 17 17:21:29 2021	(00:00)
par4206	pts/9	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:21:48	2021	- Wed Feb 17 17:21:48 2021	(00:00)
par4206	pts/10	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:22:04	2021	- Wed Feb 17 17:22:04 2021	(00:00)
par4206	pts/10	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:23:41	2021	- Wed Feb 17 17:23:52 2021	(00:00)
par4206	pts/10	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:24:08	2021	- Wed Feb 17 17:24:27 2021	(00:00)
par4206	pts/10	(IP1)f0c6c40d2f39dded0eec0cdc2e894c81	-	Wed
Feb 17	17:24:35	2021	- Wed Feb 17 18:54:59 2021	(01:30)

INFO: Tipologia adreces IP encriptades:

- (IP1) IP publica UPC (per ex. aules informatiques) [147.83.xxx.xxx]
- (IP2) IP privada UPC (per ex. eduroam, xarxes NAT, etc.) [10.xxx.xxx.xxx]
- (IP3) IP externa UPC (tambe pot estar darrera de NAT externa, per ex. biblioteques, etc.)

Detall treballs executats a nodes de calcul (no es compta boada-1):

JobID	State	NodeList	Start
End	User ExitCode	CPUTime	
60950	COMPLETED	boada-2	2021-02-17T18:14:04
2021-02-17T18:14:08	par4206	0:0	00:01:36
60963	COMPLETED	boada-2	2021-02-17T18:19:58
2021-02-17T18:19:59	par4206	0:0	00:00:24
60993	COMPLETED	boada-3	2021-02-17T18:30:53
2021-02-17T18:30:57	par4206	0:0	00:01:36
60995	COMPLETED	boada-3	2021-02-17T18:30:57
2021-02-17T18:31:02	par4206	0:0	00:02:00
60996	COMPLETED	boada-3	2021-02-17T18:31:02
2021-02-17T18:31:06	par4206	0:0	00:01:36
60997	COMPLETED	boada-3	2021-02-17T18:31:06
2021-02-17T18:31:10	par4206	0:0	00:01:36
61015	COMPLETED	boada-2	2021-02-17T18:35:15
2021-02-17T18:35:19	par4206	0:0	00:01:36
61024	COMPLETED	boada-3	2021-02-17T18:40:31
2021-02-17T18:40:33	par4206	0:0	00:00:48
61026	COMPLETED	boada-2	2021-02-17T18:41:25
2021-02-17T18:41:26	par4206	0:0	00:00:24
61029	COMPLETED	boada-2	2021-02-17T18:41:38
2021-02-17T18:41:39	par4206	0:0	00:00:24

Aquí veurem les dades de inici i fi de sessió. Pel que fa la IP, veurem un valor de hash (a l'exemple 66a931b15899a644c42ca3819450a566 i b9048c11e3692e86bdca1fe835985793) que en cas de coincidència significa que la direcció IP de connexió ha estat la mateixa per a

cada sessió corresponent. A més veiem a l'exemple que estan identificades amb el tipus IP3, per tant corresponent a IP's externes a la UPC. També hi veurem els treballs executats a les cues (no es té en compte els nodes boada-6, 7 i 8, atès que són els nodes d'entrada i on s'executa el job interactiu).

Si voleu resumir a sols les dades de IP (posarem el paràmetre -i):

```
boada-1:~/LOG-par$ ./showUsage.sh -u par4206 -i

IMPORTANT: les dades son actualitzades cada 5 minuts! Les connexions
inferiors a 5 minuts respecte l'hora actual poden no apareixer encara!

Llistat de IP encriptades des d'on s'ha connectat l'usuari par4206 des
de Feb-2021 fins Feb-2021:

(IP1) 7a54ad6cb280f0184c0a15bc66a8ffe0
(IP1) f0c6c40d2f39dded0eec0cdc2e894c81
(IP1) f0c6c40d2f39dded0eec0cdc2e894c81
```

on sols veurem les dades de IP i filtrades per duplicats.

També podrem mostrar un resum dels treballs executats:

```
boada-1:~/LOG-par$ ./showUsage.sh -u par4206 -j

...

Resum nombre de treballs executats a nodes de calcul (no es compta
boada-1): 10 treballs completats      0 treballs en altres estats
```

 També podem usar els paràmetres -i i -j a la vegada per veure les connexions i els treballs de forma resumida.

 **És important tenir en compte que les adreces IP de connexió poden estar darrera de NAT's externs i interns de mode que una mateixa IP pública estigui servint a diferents IP's privades, i per tant aquesta IP xifrada no identificaria a un únic usuari. La interpretació de les dades les deixem sota el vostre criteri.**

Per tal de posar en marxa el sistema, necessitarem crear un directori al vostre compte, on s'aniran guardant les dades de connexions i treballs executats del quadrimestre. Aquest directori serà important que no tingui accés ningú més que l'usuari per no mostrar aquestes dades a comptes d'estudiants. El directori serà /scratch/nas/1/PDI/{usuari}/LOG-{assignatura}. A més, trobareu en aquest directori el script de consulta (showUsage.sh), que us permetrà fer el filtratge de les dades per usuari.

 En cas d'estar interessats cal que ho notifiqueu al LCAC.